Cost-Efficient Segmentation under a Compute Budget

CSL7360: Computer Vision — Assignment 2 (Track A: Classical)

Lakshya Jain (B23CS1032)

November 2026

1 Objective and summary

The task is to label every pixel of a 256×256 photo as *main object* or *background*, using at most 50 ms of single-core CPU time per photo on average. This is **Track A**: cluster the image into compact regions, learn a probability that each region is object, and recover the labelling as the minimum cut of a graph over those regions. The clustering, the region model, the max-flow solver and both metrics are implemented from scratch with NumPy and OpenCV primitives only.

The pipeline runs in 23.62 ms per photo with a 6,238-byte parameter file, reaching CES = 0.4891 on DUTS-TE.

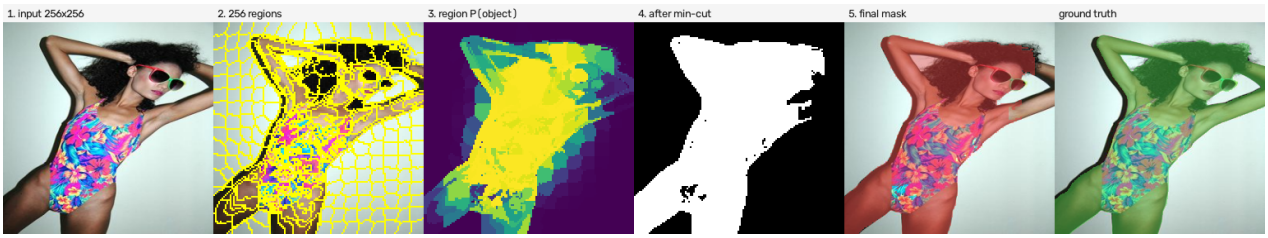


Figure 1: The pipeline on one validation image. The 256^2 input is downscaled to 128^2 and clustered into 256 compact regions; a logistic regression over 17 region cues gives P_p; the min-cut of the region graph selects the object side; the region labelling is expanded back to 256^2. Red is the prediction, green the ground truth.

2 Data

Splits. DUTS-TR (10,553 photos) is split **80/20** into 8,442 training and 2,111 validation photos with a fixed seed. All design choices — features, clustering resolution, λ and σ_d — were made on the validation split. The colour and position statistics and the logistic regression were fitted on a 3,000-photo subset of the training split, giving ≈768k labelled regions for 170 parameters. **DUTS-TE (5,019 photos) was never used for training, tuning, or any stopping decision**; it was evaluated exactly once, at the end.

Resizing. Photos are resized to 256×256 with `cv2.INTER_AREA` and masks with `cv2.INTER_NEAREST`; a mask pixel is object iff its value is ≥ 128. All metrics are computed on these 256×256 masks.

Captured photos. 8 photos taken around the campus/hostel, masked by hand on the resized image. Together they cover all five listed conditions: indoor and outdoor scenes, daylight and tube light, near and far objects, cluttered backgrounds, and partial occlusion (Section 7).

3 Method

3.1 Clustering into regions

Each pixel of the downscaled 128×128 image is described by (L, a, b, x, y), with the spatial coordinates scaled by m/S where $m = 10$ is a compactness constant and S the grid spacing, which sets the trade-off between colour and spatial distance. Centres are initialised on a regular 16×16 grid and refined by 4 rounds of Lloyd's algorithm.

Plain Lloyd's algorithm compares every pixel to every centre, costing $O(NK)$ per sweep — about 4.2M distance evaluations here. Following SLIC, a pixel is compared only against the grid centres nearest to it: the four nearest are determined by which quadrant of its cell the pixel occupies, so assignment costs $O(4N)$, *independent of* K. The restriction also keeps clusters spatially compact and connected, which the graph cut requires since it treats each cluster as a single node.

3.2 Region descriptors

For each region I compute 17 cues, all from vectorised `np.bincount` reductions:

- **Learned colour likelihood.** A 512-bin Lab histogram of object and background pixels is accumulated over the training subset; the per-region feature is the mean of $\log P(c \mid \text{obj}) - \log P(c \mid \text{bg})$ over its pixels.
- **Learned position prior.** A 16×16 map of $P(\text{object} \mid \text{position})$ averaged over the training subset, again averaged over the region.
- **Border contact** (fraction of the region's pixels on the image border), area fraction, centroid offsets $|c_x - \frac{1}{2}|, |c_y - \frac{1}{2}|$.
- **Contrast:** to the global mean colour, to the mean border colour, to adjacent regions weighted by shared border length, and a spatially weighted region-contrast term at two spatial scales.
- **Diffused backgroundness.** Border regions are usually background, and that evidence propagates to regions joined to them through colour-similar neighbours. Five rounds of similarity-weighted averaging against the border seed approximate a random walk on the region graph.
- Mean L, a, b and within-region colour spread (a flatness/texture proxy).

3.3 Region probabilities

P_p comes from a logistic regression trained from scratch with Adam on a **quadratic basis**: the 17 cues plus all pairwise products and squares, giving 170 weights. The problem stays convex, while the products let the model represent interactions between cues that a linear surface over the 17 raw features cannot. Targets are *soft* — the fraction of each region that is object — so regions straddling the boundary only partially pull the weights. Because of the basis expansion the individual weights are not interpretable in isolation, so no single cue is reported as dominant. Feature standardisation is folded back into (w, b) after fitting, so inference is a single dot product. Finally P_p is clipped to $[0.001, 0.999]$.

3.4 Graph cut

I build a graph with one node per region plus a source (object) and sink (background). Following the specification, the source connects to region p with capacity $-\log(1 - P_p)\,|p|$ and p connects to the sink with capacity $-\log P_p\,|p|$, where $|p|$ is the region's pixel count — so the unary term is a sum over pixels, commensurate with the pairwise term which is a sum over boundary pixel pairs. Cutting the sink edge leaves p on the source side, labelling it object and paying $-\log P_p$.

Two regions are adjacent if any of their pixels are 4-neighbours; they are joined in both directions with weight $\lambda W_{pq}\, B_{pq}$, where B_{pq} is the shared border length and

$$W_{pq} = \exp\!\left(-\frac{\|f(p) - f(q)\|^2}{2\sigma_d^2}\right), \qquad \sigma_d^2 = \texttt{sigma_mult} \cdot \overline{\|f(p) - f(q)\|^2} + 10^{-6},$$

with f the region mean colour and the average taken over that image's adjacent pairs. So similar neighbours are expensive to separate. Tuning on validation gave $\lambda = 32$ and $\texttt{sigma_mult} = 0.15$.

Because the pairwise term is a Potts penalty it is submodular, so the binary labelling that minimises this energy is recovered *exactly* as a minimum s–t cut, rather than approximately as with a greedy or thresholding scheme.

Max-flow. I implement **Dinic's algorithm** rather than Edmonds–Karp. Both are augmenting-path methods, but Edmonds–Karp augments one shortest path per BFS ($O(VE^2)$), whereas Dinic's groups all shortest augmenting paths of a given length into one phase: a BFS builds a level graph and a single DFS saturates a blocking flow within it. The shortest s–t distance strictly increases per phase, giving $O(V^2E)$. Measured over 300 DUTS-TE photos these region graphs take a mean of 8.8 phases (p5–p95: 4–15), against hundreds of individual augmentations for Edmonds–Karp. Capacities are scaled to integers so termination is exact — with floats, repeated subtraction of near-zero residuals can stall the search. The implementation was verified against a reference Edmonds–Karp on **2000 random graphs** (identical flow value on all of them), and the min-cut value was checked to equal the max-flow value on every test.

4 Staying inside the budget

Clustering and max-flow over all 65,536 pixels in Python cannot fit in 50 ms, so the expensive stages run on the 128^2 downscale and on the 256-node region graph; only the final nearest-neighbour upsample touches 256^2. Three measurements drove the remaining design choices.

Stage profile. Max-flow accounts for a fifth of the runtime and clustering for nearly half — the opposite of what the algorithmic complexities suggest, because the cut runs on a few hundred nodes while clustering runs on 128^2 pixels.

Table 1: Stage breakdown, 200 DUTS-TE photos. Per-stage instrumentation inflates the total by ≈15% relative to the clean measurement of 23.62 ms; the proportions are what matter.

stage	ms	%
downscale + BGR→Lab	0.24	0.9
grid-constrained K-Means	12.90	46.8
region descriptors + adjacency	8.32	30.2
logistic regression	0.31	1.1
graph build + Dinic min-cut	5.61	20.4
upsample to 256^2	0.18	0.6
total	**27.56**	**100.0**

Two optimisations. Restricting K-Means to the 4 nearest grid centres rather than the full 3×3 neighbourhood, and dropping the constant $\|f\|^2$ term so the distance reduces to a dot product, took clustering from 27.6 ms (9 candidates, 6 iterations) to 12.2 ms (4 candidates, 5 iterations), both measured at 112^2. Computing the region-contrast pairwise distances through the $\|a\|^2 - 2a{\cdot}b + \|b\|^2$ expansion — a small GEMM instead of an $(M, M, 3)$ broadcast — recovered a further 4.8 ms. The freed budget was spent on the contrast and diffusion cues.

Oracle ceiling. Labelling each region by its *true* majority upper-bounds any region-level method at a given granularity, separating clustering error from region-model error.

Table 2: Region-oracle ceiling on 300 validation photos.

clustering	mIoU	BF1	Q
96^2, 144 regions	0.9447	0.7427	0.8841
128^2, 256 regions	0.9577	0.8047	0.9118
160^2, 400 regions	0.9662	0.8531	0.9323
192^2, 576 regions	0.9713	0.8874	0.9461

At 128^2 the ceiling is $Q = 0.912$ while the model achieves 0.6512 on validation — so granularity is not the binding constraint — the region model is. Moving to 192^2 would raise the ceiling by 0.03

while roughly doubling the runtime, so the resolution was fixed at 128^2 and the remaining budget spent on region cues.

5 Metrics

Both metrics are implemented from scratch.

mIoU. One 2×2 confusion matrix is accumulated over the whole set, then IoU is computed for background and for object and the two averaged.

Boundary F1. A boundary pixel is an object pixel with a background 4-neighbour, with pixels outside the image counted as background (so a zero-padded shift is the right comparison). The 5×5 tolerance test is a binary dilation, but `cv2.dilate` is not permitted — instead I convolve the binary boundary map with a 5×5 box of ones using `cv2.filter2D`, which counts how many boundary pixels fall in each window; testing count > 0 is exactly dilation by a 5×5 structuring element. This replaces the per-pixel Python loop the brief warns against and runs over all 5,019 images in seconds. BF1 is computed per image and averaged; it is 1 if neither mask has boundary pixels and 0 if only one does.

6 Results

Measurement conditions. Timing follows Section 2b exactly: all thread environment variables set to `"1"` before importing NumPy/OpenCV, plus `cv2.setNumThreads(1)` and `torch.set_num_threads(1)`; one warm-up run discarded; then the 5,019 DUTS-TE photos segmented one at a time, summing `time.process_time()`. The timed region starts with a 256×256 photo in memory and ends with its 256×256 0/1 mask, including the downscale and colour conversion. **CPU: Intel Core i5-1340P** (laptop, single thread, no subprocesses).

Table 3: Main results. $Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1}$; $\mathrm{CES} = Q\left(1 - 0.20\frac{T_{\mathrm{CPU}}}{50\,\mathrm{ms}} - 0.10\min(1, \frac{S_{\mathrm{disk}}}{4\,\mathrm{MiB}})\right)$.

set	n	mIoU	BF1	Q	CES
validation (DUTS-TR split)	2111	0.7305	0.4660	0.6512	0.5896
DUTS-TE	5019	0.6115	0.3737	0.5402	**0.4891**
captured photos	8	0.7351	0.4024	0.6353	0.5752

$T_{\mathrm{CPU}} = 23.62$ ms (median 23.42, p99 31.91) over 5,019 photos $\quad S_{\mathrm{disk}} = 6{,}238$ bytes $= 0.006$ MiB

time penalty 0.0945 $\quad$ size penalty 0.00015 $\quad$ combined multiplier 0.9054

At 23.62 ms the time penalty removes 9.4% of Q and the 6,238-byte parameter file removes 0.015%. Q therefore dominates CES, so optimisation effort went to the region model rather than to further reducing runtime.

6.1 Validation does not predict test here

Validation $Q = 0.6512$ but DUTS-TE $Q = 0.5402$. This is not overfitting in the usual sense: the model has 170 parameters fitted on ∼768k regions. The cause is a distribution shift between the two halves of DUTS.

Table 4: Object size differs sharply between DUTS-TR and DUTS-TE.

	mean object fraction	median	photos with object $< 5\%$ of frame
DUTS-TR (validation split)	0.271	0.256	4.7%
DUTS-TE	0.148	0.109	22.2%

DUTS-TE objects are roughly **half the size** and tiny objects are nearly **five times more common**. The captured set is an independent check on this reading: its objects average 21.2% of the frame — much closer to DUTS-TR than to DUTS-TE — and its score ($Q = 0.6353$) lands next to validation (0.6512) rather than to DUTS-TE (0.5402), despite a different camera, different scenes and a different annotator — so object scale tracks the score more closely than domain does.

The position prior, the area term and the diffused backgroundness all encode an assumption of one reasonably large, roughly central object, which is what DUTS-TR contains. The damage is almost entirely in the object class: background IoU barely moves (0.8393 → 0.8326) while object IoU falls sharply (0.6218 → 0.3904), the signature of a model that under-segments small objects, matching the failure modes in Section 8. A validation split drawn from DUTS-TR therefore overestimates DUTS-TE performance for any method carrying a size or centrality prior; size-stratified validation would have shown this earlier.

7 Captured photos

Table 5: Per-photo results on the captured set. Green is the hand-drawn ground truth, red the prediction.

photo	mIoU	BF1	object % of frame
backpack	0.672	0.388	19.1%
bottle	0.606	0.347	17.0%
chair	0.857	0.635	55.3%
container	0.679	0.185	13.5%
motorcycle	0.719	0.445	24.9%
plant_doorway	0.683	0.328	11.8%
plant_ledge	0.898	0.629	12.0%
pot_outdoor	0.683	0.263	16.1%

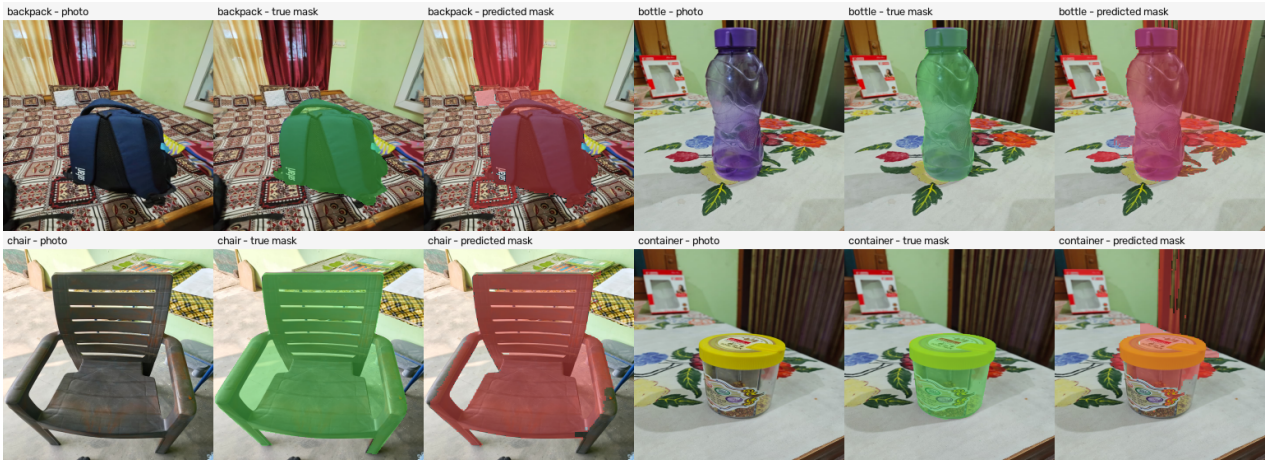


Figure 2: Captured photos 1–4. Each triple is input, true mask (green), predicted mask (red), as semi-transparent overlays.

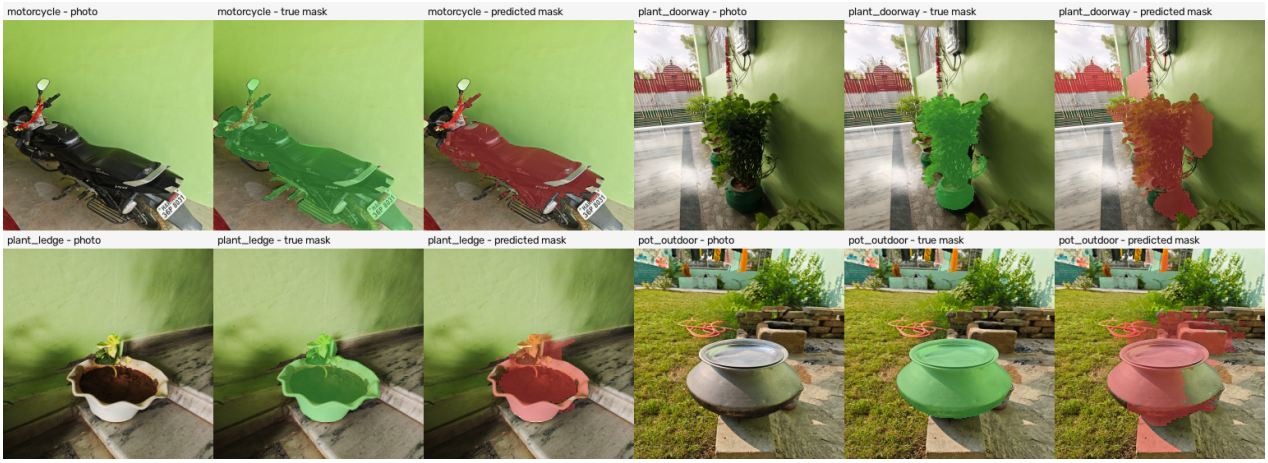


Figure 3: Captured photos 5–8, same layout.

8 Failure cases



Figure 4: Validation extremes: the three worst cases by IoU (first row and left of the second), then the three best.

1. Small or distant objects are lost entirely. When the object covers only a few percent of the frame it spans few regions, each carrying little unary weight because the terminal capacities scale with pixel count $|p|$. The pairwise term, which scales with shared border length, then overwhelms them and the cut absorbs the object into the background. This is the dominant failure on DUTS-TE and directly explains the object-IoU collapse in Section 6.1: 22% of DUTS-TE photos are in this regime versus under 5% of validation photos.

2. Low contrast against a patterned surface. `container` is the worst captured photo by boundary accuracy (BF1 0.185 against a set mean of 0.4024). A translucent tub sits on a densely floral tablecloth: the colour likelihood and every contrast cue see cloth patches that differ from their surroundings as much as the tub does, so P_p is high for both and the prediction spreads outward across the cloth. `bottle` fails similarly (mIoU 0.606, lowest of the eight): the bottle is translucent, so the tablecloth shows through and the region colours inside the object match the background.

3. Objects merging with a similarly coloured support. In `pot_outdoor` (BF1 0.263) a grey metal pot rests on a grey stone slab in shadow. Region mean colours either side of the true boundary are nearly identical, so $W_{pq} \approx 1$ there and the pairwise term makes cutting between them expensive. The smoothness prior behaves as designed; the difficulty is that the strongest edge in the image is a brightness step inside the pot rather than at its outline. The mask absorbs part of the slab.

4. Thin and fragmented structure. `plant_doorway` (BF1 0.328) is foliage with sky and wall showing between the leaves. At 128^2 the gaps are smaller than one region, so no clustering at this resolution can represent them; the prediction becomes a ragged blob. This is a granularity failure rather than a model failure — the one failure mode the oracle study in Section 4 says more resolution would genuinely fix.

5. Objects touching the image border. Border contact is one of the strongest background cues, so an object running off the frame has its border-touching regions actively pushed towards background and the mask is clipped along the edge.

6. Boundary quality is the weaker half of Q throughout. BF1 (0.3737) trails mIoU (0.6115) on every set. Region boundaries are quantised to the 128^2 grid, giving a $256/128 = 2$ px step — right at the 5×5 (± 2 px) matching tolerance, so a region edge that is slightly misplaced falls outside it. The oracle study shows the headroom is real: a perfect region labelling at this resolution scores BF1 0.80, so most of the loss is the region model misplacing boundaries, not the grid. Note the spread across the captured set is wide — `chair` (0.635) and `plant_ledge` (0.629) are large, well-separated objects against plain backgrounds, roughly double the BF1 of the cluttered cases.

9 Reproducing

```
python src/build_cache.py     # resize DUTS into a 256x256 cache
python src/train.py           # stats, features, logreg, tune lambda/sigma
python src/retune.py          # widen the lambda/sigma grid
python src/evaluate.py        # val + DUTS-TE + captured, and timing
python src/figures.py         # report figures
python predict.py --image photo.jpg --out mask.png
```

`predict.py` prints exactly the two required lines, `T_CPU_ms:` and `S_disk_bytes:`. It times 20 runs on the single photo given to it, so its reported time varies with that photo's content (images with larger objects produce more regions and a longer cut); the 23.62 ms figure above is the mean over all 5,019 DUTS-TE photos, which is what the CES definition uses.